

Extended Abstract

Motivation Experience replay buffers are fundamental to reinforcement learning, allowing agents to revisit past transitions and improve sample efficiency. Prioritized Experience Replay (PER, Schaul et al. (2016)) enhances this by sampling informative transitions based on temporal-difference (TD) error, demonstrating significant improvements in single-task settings. However, multi-task reinforcement learning (MTRL) presents unique challenges that existing replay mechanisms fail to address. The key challenge lies in determining how to prioritize transitions not only based on their individual learning signal but also considering task relationships and cross-task dynamics. Current approaches like Task-Aware Prioritized Sampling Lin et al. (2024) show limited scalability and lack theoretical guarantees for performance, motivating our development of MTRL-native strategies.

Method We develop three novel sampling strategies for multi-task experience replay, drawing inspiration from gradient alignment, algorithmic fairness, and robust optimization literature. First, we explore gradient-based sampling where task similarity is quantified using cosine similarity between task-specific gradients: $\text{Sim}(j, k) = \cos(g_j, g_k) = \frac{g_j \cdot g_k}{\|g_j\| \|g_k\|}$. Second, we implement fairness-aware sampling based on accelerated momentum algorithms, where task priorities $q_t(j)$ are updated as $q_t(j) \leftarrow q_{t-1}(j) \exp(\gamma l_t(j))$, with $l_t(j)$ representing the TD error for task j at timestep t . Third, we develop robust sampling that optimizes against adversarially chosen task distributions by solving $\theta^* = \max_{\theta} \min_{p \in \Delta^K} \sum_{j=1}^K p_j \mathbb{E}_{p_{\theta}} [\sum_{t=1}^T r_j(s_t, a_t)]$ through alternating primal-dual updates.

Implementation We conduct experiments on the MetaWorld benchmark using Multi-Task 10 (MT10) and Multi-Task 50 (MT50) environments with Multi-Task Soft Actor-Critic (MT-SAC) as the base algorithm. Our experimental framework includes three baselines: uniform sampling (B1), standard PER (B2), and multi-task PER (B3). We systematically investigate four key aspects: (1) performance imbalance analysis across individual tasks to distinguish between inherent task difficulty and multi-task interference effects, (2) gradient alignment computation to assess task similarity correlations with success rates, (3) fairness-aware sampling implementation with dynamic task reweighting, and (4) robust optimization with adversarial task distribution learning. Each method is evaluated over 1000K environment steps with success rate metrics computed across random seeds.

Results Our baseline analysis reveals that multi-task PER (B3) achieves the best performance with 63% average success rate on MT10 compared to 56% for uniform sampling. Task-level analysis reveals that single-task SAC achieves up to 50% success on the three most difficult tasks compared to 24-31% in multi-task settings, indicating multi-task interference rather than inherent task difficulty. Gradient alignment analysis shows minimal correlation with task performance ($R^2 = 0.02$), suggesting limited utility as a task similarity metric. Our proposed fairness-aware and robust optimization methods achieve modest improvements (47.5% vs 45.0% average success rate) with reduced variance in the robust case, but overall gains remain insufficient to warrant adoption over existing PER.

Discussion The performance gap between single-task and multi-task learning on difficult tasks confirms that task interference, rather than inherent complexity, drives poor performance in MTRL settings. The minimal correlation between gradient alignment and task performance indicates that shared feature representations in task-specific heads may obscure meaningful gradient differences. While our fairness-aware and robust methods show theoretical promise, the modest practical improvements suggest that either more aggressive reweighting is needed or fundamental limitations exist in leveraging replay buffer-based approaches to improve multi-task learning. The persistent zero minimum success rates across all methods indicate that some tasks may require fundamentally different prioritization strategies or architectural modifications beyond experience replay improvements.

Conclusion This work presents a systematic investigation of experience replay strategies for multi-task reinforcement learning. While our proposed methods demonstrate theoretical soundness and modest empirical improvements, they fall short of providing substantial performance gains. We also contribute by identifying task interference as a primary bottleneck in MTRL and establishing that gradient-based similarity metrics provide insufficient signal for effective task prioritization. Future work should explore alternative task similarity measures, investigate sequential multi-task settings where temporal task relationships may provide stronger signals, and develop theoretical frameworks with convergence guarantees for robust multi-task optimization across varying task weights.

Improving Experience Replay in Multi-Task RL

Tarun Chiruvolu
Department of Computer Science
Stanford University
tarunc@stanford.edu

Advit Deepak
Department of Computer Science
Stanford University
advit@stanford.edu

Abstract

Experience replay is a critical component of reinforcement learning (RL) systems, enabling agents to learn from past experiences and improve sample efficiency. While Prioritized Experience Replay (Schaul et al. (2016)) has demonstrated substantial improvements in single-task settings by sampling transitions based on temporal-difference error, existing replay mechanisms are not designed for multi-task RL where agents must perform well across multiple tasks using shared policies. In this work, we explore redesigned PER schemes for multi-task settings, where transition priority incorporates both task identity and task similarity. We also take inspiration from fairness-based and robust-optimization literature. We hypothesize that revisiting transitions from related tasks can enhance cross-task learning. We evaluate our approach on the MetaWorld environment using Multi-Task 10 (MT10) and Multi-Task 50 (MT50) benchmarks. Our investigation includes four key experiments: (1) characterizing performance imbalance across individual tasks, (2) analyzing gradient alignment to quantify task relationships, (3) implementing fairness-aware sampling inspired by algorithmic fairness literature, and (4) developing robust sampling methods based on adversarial task distribution optimization. While we establish rigorous baselines and observe modest improvements in average success rates through fairness-based and robust-based sampling approaches, the performance gains are not substantial enough to warrant adoption over standard PER methods. Our analysis reveals that certain tasks consistently underperform in multi-task settings compared to single-task training, suggesting interference effects rather than inherent task difficulty. Additionally, gradient alignment shows minimal correlation with task performance, while robust-sampling leads to slightly lower variance. These findings provide insights into the challenges of experience replay in multi-task RL and highlight opportunities for future research in this domain.

1 Introduction

A core component of reinforcement learning (RL) systems is the *replay buffer*, which allows agents to learn from past experiences, thereby improving learning stability and sample efficiency. However, in standard online RL, performance often suffers due to the non-i.i.d. nature of visited states: since replay buffers typically store a stream of recent transitions, older yet valuable experiences are frequently forgotten. This can degrade the agent’s ability to generalize or recover from suboptimal behavior.

To address this challenge, *Prioritized Experience Replay* (PER) (Schaul et al. (2016)) has been proposed as a mechanism for adaptively sampling transitions from the buffer. By assigning sampling probabilities based on the temporal-difference (TD) error, PER ensures that more informative transitions—those that are expected to have a higher learning signal—are revisited more frequently. While PER has demonstrated substantial improvements in single-task settings, no existing replay buffer variants are designed to account *multi-task reinforcement learning* (MTRL) settings.

In MTRL, agents must perform well across multiple tasks, ideally using a shared policy that facilitates transfer of knowledge between tasks. This introduces a new frontier for experience replay: *how can we redesign PER for multi-task RL to account for both task-specific and cross-task dynamics*.

In this work, we explore redesigned PER schemes for the multi-task setting, where the notion of transition priority is modified to incorporate task identity and task similarity. Our intuition is that if an agent performs well on one task and poorly on another semantically related task, revisiting transitions from both may enhance learning. We evaluate our method on the MetaWorld (Yu et al. (2019a)) environment on both Multi-Task 10 (MT10) and Multi-Task 50 (MT50) splits. We establish rigorous baselines and explore different approaches to defining and leveraging task similarity within the prioritization mechanism, taking inspiration from fairness and robust optimization literature.

We observe a minor increase in performance via fairness-based and robust-based sampling, however these are modest changes in performance and not significant enough to warrant usage of standard PER. We provide further explanation for these observations as well as future extensions in Section 6.

2 Related Work

In this section, we discuss prioritized experience replay (Section 2.1), its extension to the multi-task setting (Section 2.2), and finally relevant works on gradient alignment (Section 2.3.1), fairness-based optimization (Section 2.3.2), and robust-optimization (Section 2.3.3) that inspire our experiments.

2.1 Prioritized Experience Replay (PER)

Schaul et al. (2016) introduced *Prioritized Experience Replay* (PER), a modification of standard experience replay that improves efficiency by prioritizing the replay of informative transitions. Each transition i in the buffer is assigned a *priority score* p_i based on its temporal-difference (TD) error δ_i :

$$\delta_i = \left| r_i + \gamma \max_{a'} Q(s'_i, a') - Q(s_i, a_i) \right|, \quad p_i = \delta_i + \epsilon, \quad P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (1)$$

Here, $\epsilon > 0$ is a small constant added to ensure all transitions have nonzero priority. The probability of sampling transition i is $P(i)$, where α determines the level of prioritization. This allows the learning algorithm to focus on transitions with higher learning potential, as measured by TD error.

Limitation: This work is meant for single-task settings, whereas ours is specifically for multi-task.

2.2 Task-Aware Prioritized Sampling

To extend PER to the multi-task setting, Lin et al. (2024) proposed *Task-Aware Prioritized Sampling*. This method retains the transition-level priorities of PER, but also incorporates *task-level priority*, sampling based on how much each task contributes to overall learning. Each task j maintains its own buffer $\mathcal{D}_j$, and its priority P_j is proportional to the cumulative TD error over its transitions:

$$P_j = \frac{\sum_{i \in \mathcal{D}_j} \delta_i}{\sum_k \sum_{i \in \mathcal{D}_k} \delta_i} \quad (2)$$

This encourages the agent to sample more transitions from tasks with higher overall TD error (underperforming), allocating more learning capacity to areas with greater room for improvement.

Limitations: Although this formulation is more task-aware than PER, it fails to improve on the status quo in a principled manner. Indeed, the authors mention that their method improves on SAC in the MT10 setting, but this advantage disappears when scaling to a larger number of tasks MT50. Moreover, this method lacks theoretical guarantees that demonstrate lower bounds on worst task performance, whereas our fairness- and robust-optimization inspired sampling methods factor account for adversity.

2.3 Inspiration for Task-Aware Multi-Task PER

To overcome the limitations of multi-task PER, we analyze works on gradient alignment, fairness-based optimization, and robust-optimization that inspire each of our experiments (Section 3).

2.3.1 Gradient-Based Sampling via Alignment

Riemer et al. (2019) introduced a gradient alignment method for continual learning to reduce interference between tasks. Their key insight is that learning is more effective when task gradients are aligned. This is quantified using cosine similarity between gradients of loss from different tasks:

$$\text{Sim}(j, k) = \cos(\mathbf{g}^j, \mathbf{g}^k) = \frac{\mathbf{g}^j \cdot \mathbf{g}^k}{\|\mathbf{g}^j\| \|\mathbf{g}^k\|} \quad (3)$$

where $\mathbf{g}^j$ is the gradient of the loss on a sample from task j . In our multi-task RL setting, we can adapt this approach to estimate *task similarity* and use it to guide experience replay. Specifically, tasks with more aligned gradients can be prioritized when sampling, enabling more effective transfer of shared structure while avoiding destructive interference when learning across different tasks.

Limitation: This paper deals solely with meta- and continual learning and does not extend their gradient alignment method to multi-task reinforcement learning setting or experience replay buffers.

2.3.2 Algorithmic Fairness in Machine Learning

The first approach we use to build a better multi-task PER is based on existing literature in algorithmic fairness. Specifically, there exists a body of literature focusing on fairness among subgroups in machine learning - for instance, Abernethy et al. (2022) attempt to improve error on the worst subgroup. They specifically focus on classification tasks like facial recognition, where there naturally exists a definition of subgroups based on, e.g., ethnicity. This can be naturally extended to the case of multi-task RL - each task is a subgroup on which some performance metric (TD error, total return) needs to be optimized, which additionally elides the messiness of defining subgroups in classification.

Limitation: This paper works solely with classification tasks and does not investigate MTRL settings.

2.3.3 Learning Worst-Case Distribution Over Tasks

A further approach to ameliorate performance imbalance is to extend beyond the case of maximizing worst-task performance. In Bai et al. (2025), the authors apply this logic to the case of federated learning, where the goal is to minimize loss over an adversarially chosen distribution over clients. At each iteration, both a performance metric and weighting over clients are optimized, based on connections with Lagrange multiplier theory. Our goal is to both simultaneously optimize this worst case loss and use this worst case per-iteration weighting as task weights for sampling.

Limitation: This paper examines tasks including regression and classification, but not MTRL.

3 Method

In this section, we establish baseline methods (Section 3.1) and discuss our experiments (Section 3.2).

3.1 Baselines

In addition to sampling experiences uniformly (B1-Uniform), we implement the methods proposed in *Prioritized Experience Replay* (B2-PER) Schaul et al. (2016) and *Task-Aware Prioritized Sampling* (B3-MT-PER) Lin et al. (2024) to serve as three baselines. Concretely, we define them as follows:

- (B1-Uniform) Replay buffer with uniform sampling, where a fixed number of transitions are sampled per update iteration, uniformly from the set of all transitions across all tasks.
- (B2-PER) Replay buffer with prioritized sampling based on transitions' temporal-difference error (TDE) from the set of all transitions across all tasks (proposed in Schaul et al. (2016)).
- (B3-MT-PER) Replay buffer with multi-task prioritized sampling, which prioritizes sample based on a function of task-level and TDE-level priority (proposed in Lin et al. (2024)).

3.2 Experiments

We systematically investigate experience replay to develop improved sampling methods. Specifically,

- **Experiment 1:** Characterize performance imbalance across individual tasks. *Are there some tasks that systematically underperform? What is the upper bound success rate?*
- **Experiment 2:** Analyze gradient conflict/alignment to quantify discrepancies between tasks. *Is gradient alignment an indicator of task difficulty?*
- **Experiment 3:** Develop and implement new fairness-aware sampling to ensure more persistent task priorities. *Does changing task weights dynamically improve performance?*
- **Experiment 4:** Develop and implement new robust sampling. *Does optimizing with respect to an adversarially chosen distribution lead to greater generalizability?*

3.2.1 Performance Imbalance Across Tasks

Before implementing and benchmarking new experience replay strategies, we aim to better understand multi-task learning by quantifying performance imbalance across individual tasks in MT10 and MT50.

For each of our three baselines (B1-Uniform, BT-PER, B3-MT-PER), we will not only track overall success rate, but also individual task success rate across 8 random seeds. This will help quantify varying difficulty across tasks and enable us to better compare results with our proposed methods.

Next, for the 3 tasks with the lowest individual success rate, we run single-task SAC on that specific-task via the MT1 environment to examine performance. We aim to answer if these difficult tasks are fundamentally hard, or if the multi-task setting is producing an impediment to learning.

3.2.2 Gradient-Alignment for Task Similarity

Our first new experience replay strategy will use gradient alignment instead of the ratio of cumulative task temporal-difference error to determine task priority. Mathematically, this formulation looks like:

$$P_j = \frac{g^j \cdot (\sum_k g^k)}{\|g^j\| \|(\sum_k g^k)\|}$$

To determine if this sampling strategy will improve performance, we will first compute and log the cosine similarity of each task’s average transition gradient with respect to the mean gradient across all tasks and their transitions. Next, we will compute the correlation of this similarity with the individual task success rates from Experiment 1. This will answer the question of whether gradient alignment (direction correlatedness) is an indicator of task difficulty.

Next, if we observe a correlation, we will implement this sampling strategy in the replay buffer and re-learn on MT10 and MT50. We chose to split the experiments this way since each learning (1000K environment steps) is expensive (see Section B), especially when computing and storing gradients, while logging serves as a quick way to check whether this approach bears any valuable signal.

3.2.3 Implementing Fairness-Aware Sampling

In Abernethy et al. (2022), the authors propose an accelerated momentum-based algorithm to improve performance on worst-case subgroups. We modify this algorithm to work in the multitask setting. If we $l_t(j)$ be the TD-error of the model on task j at timestep t , we update the task priorities $q_t(j)$ by multiplying them by $\exp(\gamma l_t(j))$, where $\gamma > 0$ is a hyperparameter to tune. In algorithm 1, we provide further details - the task priorities (after normalization) are used in the same way as in multi-task PER - if we have B total transitions in a batch, $q_t(j) \cdot B$ are used for task j . Since the authors prove certain fixed-point guarantees for their method, our goal was that this method would be more theoretically principled and ultimately perform better.

3.2.4 Learning Adversarial Task Distribution

The next direction we focus on is shifting from optimizing the worst task performance to optimizing against an arbitrarily chosen task distribution. More formally, we would like to optimize the following objective:

$$\theta^* = \max_{\theta} \min_{p \in \Delta^K} \sum_{j=1}^K p_j \mathbb{E}_{p_{\theta}} \left[\sum_{t=1}^T r_j(s_t, a_t) \right]$$

Algorithm 1 Active Sampling-Based Experience Replay

Require: step size $\eta > 0$, momentum parameter $\gamma > 0$,

- 1: number of tasks K , epochs T , batch size B
 - 2: **Init:** $\mathbf{P}_0 = (\frac{1}{K}, \dots, \frac{1}{K})$, $\theta_1 \in \Theta$ arbitrary, $\nabla_0 = 0$
 - 3: **for** $t = 1 \dots T$ **do**
 - 4: **Compute** $l_t(j) \leftarrow \|Y_{t,j} - Q_{\theta_t}(s, a)\|_2^2$ $\triangleright j = 1, \dots, K$
 - 5: **Update** $q_t(j) \leftarrow q_{t-1}(j) \exp(\gamma l_t(j))$ $\triangleright j = 1, \dots, K$
 - 6: **Normalize** $\mathbf{q}_t \leftarrow \frac{\mathbf{q}_t}{\|\mathbf{q}_t\|_1}$
 - 7: **Compute** $B_j \leftarrow q_t(j) \cdot B$
 - 8: **Compute** $\nabla_t \leftarrow \nabla_{\theta} \sum_{j=1}^K \sum_{i=1}^{B_j} l(s_j^i, a_j^i, \theta_t)$
 - 9: **Update** $\theta_{t+1} \leftarrow \text{PROJ}_{\Theta}(\theta_t - \eta \nabla_t)$
 - 10: **return** $\bar{\theta}_T = \frac{\sum_{t=1}^T \theta_t}{T}$
-

Past work like Bai et al. (2025) uses a Lagrange multiplier-inspired method to solve the problem, with a primal update that optimizes the reward, and a dual update that optimizes for the worst case distribution. In our case, we opt for the following simpler set of updates - setting

$$\theta_{t+1} \leftarrow \theta_t + \eta \nabla_{\theta} \left(\sum_{j=1}^K p_j^t \mathbb{E}_{p_{\theta}} \left[\sum_{t=1}^T r_j(s_t, a_t) \right] \right)$$

and updating the weights

$$p^{t+1} \leftarrow p_t - \eta \nabla_{p^t} \left(\sum_{j=1}^K p_j^t \mathbb{E}_{p_{\theta}} \left[\sum_{t=1}^T r_j(s_t, a_t) \right] \right)$$

which simplifies to

$$p_j^{t+1} \leftarrow p_j^t - \eta \left(\mathbb{E}_{p_{\theta}} \left[\sum_{t=1}^T r_j(s_t, a_t) \right] \right).$$

After this, we perform a normalization step to ensure that the weights sum to 1. Then, we use these weights p^t as the task priority weights for experience replay. Due to some debugging difficulties, in our implementation, we only use the priority updates when optimizing p^t and use the original policy gradient objective for optimizing θ .

4 Experimental Setup

4.1 Environment

We conduct all experiments in the Meta-World benchmark suite, a widely used environment for evaluating multi-task reinforcement learning algorithms. Specifically, we focus on the Multi-Task 10 (MT10) and Multi-Task 50 (MT50) settings Yu et al. (2019b), which consist of 10 and 50 distinct robotic manipulation tasks respectively. These settings are designed to test the ability of agents to learn shared representations and generalize across diverse tasks, making them ideal for benchmarking.

4.2 RL Agent Policy

To train our agents, we use multi-task Soft Actor-Critic (MT-SAC) Sodhani et al. (2021), an extension of the Soft Actor-Critic (SAC) algorithm Haarnoja et al. (2018) adapted for the multi-task setting. MT-SAC maintains separate task encodings while sharing a common policy and value function backbone, enabling efficient knowledge transfer across tasks. This setup provides a strong and stable baseline for evaluating the efficacy of various experience replay strategies in multi-task scenarios.

4.3 Evaluation Metrics

To compare experience replay methods, we fix a budget of $1000K$ environment steps and report the minimum success rate (SR), average SR, and standard deviation across tasks, measured per task. We average over 2 – 8 runs per reported number, due to compute limits (discussed further in Section B).

4.4 Experimental Assumptions

Our experimental setup relies on a few assumptions that may limit the generalizability of our findings:

We assume that modifications to the experience replay sampling strategy alone can yield significant performance improvements in multi-task RL. By holding the agent architecture, hyperparameters, and environment (MetaWorld) constant, we aim to isolate the effect of sampling mechanisms. However, this assumption may underestimate the interdependence between replay strategies and other algorithmic components, potentially masking synergistic effects from joint optimization.

Our experimental setup also assumes that the relative difficulty and importance of tasks remain stationary throughout training. This assumption underlies our fairness-aware and robust optimization approaches, which adapt task priorities based on current performance. In practice, task difficulty may evolve as the agent learns, potentially invalidating static prioritization schemes.

Extending this point, we further assume that the MetaWorld benchmark provides a representative sample of multi-task RL challenges. This assumption may limit generalizability to other domains with different task structures, reward densities, or temporal dependencies. The assumption is particularly strong given that all MetaWorld tasks involve similar robotic manipulation primitives.

Finally, our comparison assumes that all methods benefit equally from the same amount of environment steps ($1000K$). This assumption may disadvantage methods and tasks that require longer to converge or that exhibit different learning dynamics, potentially masking true relative performance.

5 Results

5.1 Baseline Replay Buffers - MT10, MT50

We run each baseline for $1000K$ steps in both MT10 and MT50 and report results in Table 5.1¹.

MetaWorld Env	Metric	B1 - Uniform	B2 - PER	B3 - MT-PER
MT10	Average SR	0.56	0.58	0.63
	Std Dev SR	0.08	0.07	0.08
	Minimum SR	0.00	0.00	0.00
MT50	Average SR	0.30	0.31	0.31
	Std Dev SR	—	—	—
	Minimum SR	0.00	0.00	0.00

Table 1: Success Rate (SR) Comparison across All Baseline Experience Replay Methods

We clearly see that while adding task-agnostic prioritized experience replay somewhat improves overall success (+2% average SR from B1 to B2), the inclusion of task priority with the multi-task PER variant leads to greater improvement (+5% average SR from B1 to B3). However, across all baselines, we see that there is always at least one task with zero percent success rate, which motivates our development of fair and robust sampling methods as presented in Section 3.2.

5.2 Performance Imbalance Across Tasks

Next, we analyze the task-level success rates, motivated by the 0.00 minimum SR across all baseline methods. We analyze the 8 random runs in MT10² and present average SRs per task in Table 5.2.

¹Each run of MT50 takes ~ 110 hours, so we were unable to run several seeds to establish std dev values.

²We chose to focus on MT10 only due to compute limitations as well as brevity of reporting (visualizing 10 vs. 50 tasks). MT10 serves as an upper bound, which is what we aim to understand via this experiment.

Method	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9
<i>B1</i>	0.91	0.82	0.55	0.79	0.29	0.66	0.76	0.38	0.20	0.28
<i>B2</i>	0.94	0.87	0.70	0.68	0.20	0.60	1.00	0.20	0.25	0.36
<i>B3</i>	0.97	0.86	0.45	0.90	0.23	0.83	0.99	0.35	0.37	0.36

Table 2: Per-Task Success Rates (SR) for Each Replay Method Across MT10 Tasks. Specifically, the tasks are: *T0* - *reach*, *T1* - *push*, *T2* - *pick place*, *T3* - *door open*, *T4* - *drawer open*, *T5* - *drawer close*, *T6* - *button press topdown*, *T7* - *peg insert side*, *T8* - *window open*, *T9* - *window close* (Section A.1).

We observe that there are some tasks that systematically underperform across all baselines. For instance, *T4* (drawer-open), *T7* (peg-insert), and *T8* (window-open) are the three worst-performing tasks by a considerable margin relative to the other tasks. We investigate whether the multi-task setting is leading to low performance, or whether these three tasks are fundamentally difficult.

We implement and run single-task SAC for each of the three tasks. SAC is run with 4 random seeds for 1000K steps in a single-task MetaWorld environment MT1, and results are compared in Table 3.

	T4	T7	T8
<i>Single-Task SAC</i>	0.25	0.50	0.50
<i>Multi-Task SAC</i>	0.24	0.31	0.27

Table 3: Success Rates on Select Tasks for SAC vs MT-SAC

We see that some tasks do appear to be more difficult, but they are more solvable with single-task SAC, which could indicate that the multi-task setting is the culprit for the failures, not the task itself.

5.3 Gradient-Alignment for Task Similarity

Motivated by our findings in Section 5.2, we now aim to improve performance across all tasks as Table 5.2 indicates our multi-task setting may be a bottleneck. Specifically, we investigate task similarity and whether alignment of gradients leads to better learning (learn similar tasks together).

In this experiment, we computed the cosine similarity of the gradient of the TD error on each individual task with the gradient of the TD error when averaged over tasks (the true gradient when using PER). Here, the goal was to confirm whether the intuition of gradient similarity was relevant - i.e., tasks with more uncorrelated gradients would perform more poorly. However, as seen in Figure 1, there is little correlation, and in addition, the cosine similarity is very high. This is potentially due to the fact that there is a task-specific head at the end of the actor and critic models, so the gradients for the earlier layers are likely very similar. Also, when plotting the correlation with success rate, we saw almost none (an R^2 value of 0.02) on MT10. This, however, could be due to insufficient samples and lack of training to convergence on some tasks and might require further investigation.

Given these observations, we chose not to fully train with our proposed gradient-based sampling method in Section 3.2.2 and instead focus on alternative sampling methods given limited compute.

5.4 Implementing Fairness-Aware Sampling/Adversarial Task Distribution

Next, we implemented the fairness-aware sampling from Section 3.2.3 and the learned adversarial task distribution approach from Section 3.2.4 and tested them on the MT10 environment, shown in Figure 2. Specifically, we computed average and standard deviation across the four seeds in Table 4.

Ultimately, we found little to no changes in performance relative to PER and MT-PER. We hypothesize that this could be either due to an error in implementation/insufficient hyperparameter tuning, or the fact that the updates we use are less aggressive than the updates from multi-task PER. Interestingly, we do see lower standard deviation when using robust optimization compared to the other methods.

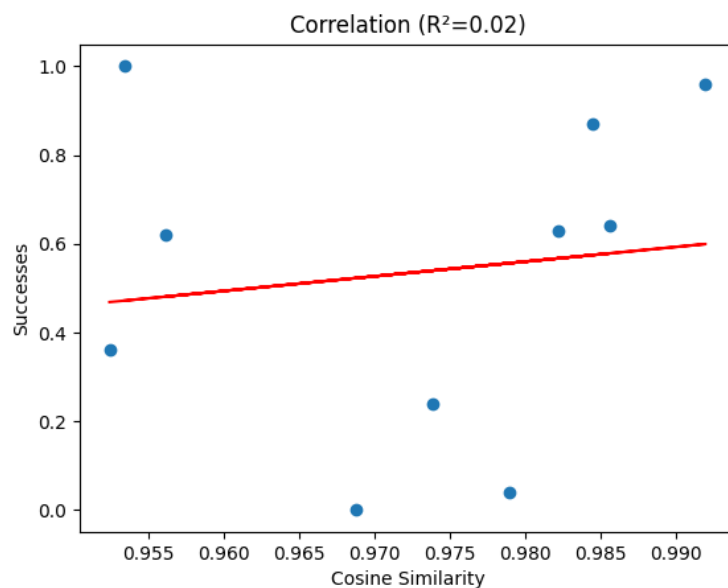


Figure 1: Gradient cosine similarity shows low correlation with task success rate.

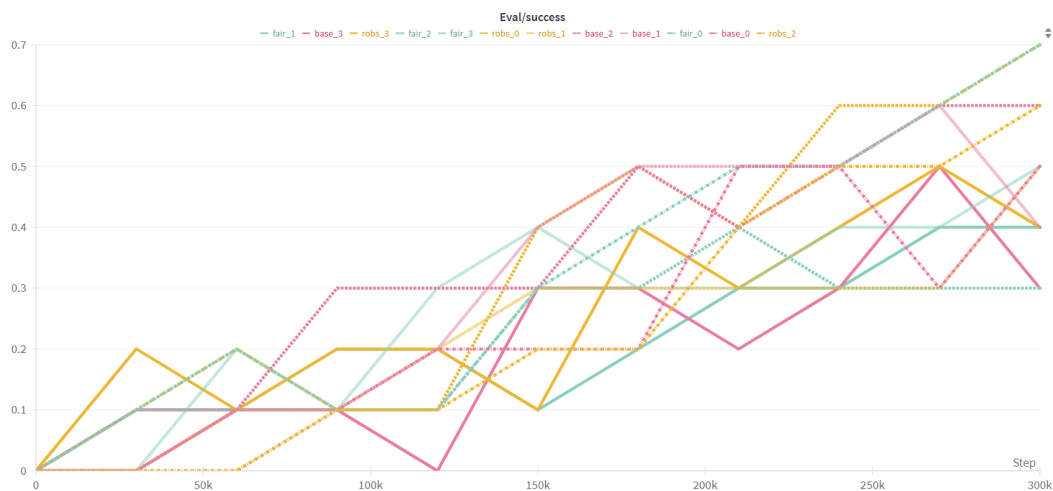


Figure 2: Eval success curves of Base (MT-PER, pink), Fair (Fairness Optimization, teal), and Robs (Robust Optimization, orange) Sampling Methods across four seeds each in MT10 environment.

	Base MT-PER	Fairness-Based	Robust Optim.
<i>Average SR</i>	0.450	0.475	0.475
<i>Std Dev SR</i>	0.111	0.147	0.082

Table 4: Success Rates on MT10 for MT-PER, Fairness-Optimized, and Robust-Optimized Variants.

6 Discussion

We discuss our experiments in this section. We conducted four ablation experiments that probe algorithmic assumptions and the stability of task success rates in multi-task reinforcement learning.

In *Experiment 1*, we investigate whether certain tasks in Meta-World are inherently more difficult and thus require consistent prioritization over multiple training epochs. As observed in Table 5.2, some tasks like *drawer-open* (T4) and *peg-insert-side* (T7) have persistently low success rates across baselines. However, when isolated and trained using single-task SAC, these tasks achieve reasonable performance (Table 3), suggesting that the failure arises from interference in the multi-task setting rather than from task-specific hardness. This validates the need for improved prioritization methods.

In *Experiment 2*, we examine whether gradient alignment (via cosine similarity) is a meaningful signal for identifying similar tasks. We find minimal correlation between gradient similarity and per-task success rates (Figure 1), with an R^2 of only 0.02. This suggests that gradient uncorrelatedness is too coarse to capture nuanced task interactions, especially given shared feature representations across task-specific heads. Thus, this metric may not serve as a reliable proxy for task difficulty.

Finally, in *Experiment 3* and *Experiment 4*, we develop and test alternative sampling methods based on fairness-aware and robustness-based literature, motivated by the extreme variance in per-task success and the persistent failure of some tasks across baselines. While these methods (Figure 2) showed marginal improvements in average success rate and a reduction in variance (Table 4), the overall gains were modest. This could be attributed to insufficient tuning, a need for more aggressive task reweighting, or limitations inherent to the replay buffer structure for overall task success.

7 Conclusion

In conclusion, we found that while our proposed modifications led to modest changes in performance, they were not significant enough to warrant usage over better-tested methods like PER. Potential areas for future work could include extending our method to different multi-task RL environments such as MTEEnv or Minigrid, or focusing on a setting where tasks can be programatically generated. Doing so would enable us to determine whether our method failures were due to inherent difficulties in MetaWorld or a more general logical issue with our approach. Another potential area of improvement could be to extend our line of work to the sequential multi-task/meta-RL setting. Our work focuses on the case where we would like to maximize average reward, so switching to this situation would also allow us to investigate the value of task similarity in experience replay, as it might be more valuable to increase the priority of older, more similar tasks as new tasks are presented. A separate line of inquiry would also be to provide proofs of convergence/performance bounds for the robust RL setting, since one of the main motivations for switching from the multi-task PER method was the theoretical guarantees enjoyed by the fairness and robust optimization literature. Ultimately, however, we found our results to be inconclusive regarding the benefits of our modified procedures.

8 Team Contributions

- **Tarun Chiruvolu:** (1) Setup multi-task MetaWorld environment and implement MT-SAC RL policy, (2) Lead Experiment 2 on Gradient-Alignment for Task Similarity, (3) Lead Experiment 3 on Fairness-Aware Sampling, (4) Co-Lead Poster and Project Report
- **Advit Deepak:** (1) Train agent without PER, with PER, and with MT-PER as baselines, (2) Lead Experiment 1 on Performance Imbalance across MT10 & MT50 Tasks, (3) Lead Experiment 4 on Adversarial Task Distributions, (4) Co-Lead Poster and Project Report

Changes from Proposal We made a few adjustments from our original breakdown in our Project Proposal. Namely, we added two more baselines (PER, MT-PER) and changed our project direction.

Our original hypothesis was that quantifying the similarity between tasks via task-specific embeddings would enable better priority sampling. However, we found it difficult to define an adequate metric for task similarity. Moreover, we found that most existing approaches involved training an additional embedding model or using some other representation-learning based approach, which would be difficult with our resource constraints. After discussions with Professor Finn, we decided to pivot.

We revised our experiments to instead improve task-level priority. Lin et al. (2024) denoted a formulation that defines a task’s priority as the ratio of its transitions’ cumulative TD error to that of all tasks. Instead, our Experiments 2 – 4 use techniques from gradient alignment, fairness-aware, and robust-optimization to redefine task priority and improve overall multi-task learning (Section 5).

References

- Jacob Abernethy, Pranjali Awasthi, Matthäus Kleindessner, Jamie Morgenstern, Chris Russell, and Jie Zhang. 2022. Active Sampling for Min-Max Fairness. arXiv:2006.06879 [stat.ML] <https://arxiv.org/abs/2006.06879>
- Wen Bai, Yi Wong, Xiao Qiao, and Chin Pang Ho. 2025. Distributionally Robust Federated Learning: An ADMM Algorithm. arXiv:2503.18436 [cs.LG] <https://arxiv.org/abs/2503.18436>
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. (01 2018). <https://doi.org/10.48550/arXiv.1801.01290>
- Po-Shao Lin, Jia-Fong Yeh, Yi-Ting Chen, and Winston H. Hsu. 2024. Shared-unique Features and Task-aware Prioritized Sampling on Multi-task Reinforcement Learning. arXiv:2406.00761 [cs.LG] <https://arxiv.org/abs/2406.00761>
- Matthew Riemer, Ignacio Cases, Robert Ajemian, Miao Liu, Irina Rish, Yuhai Tu, and Gerald Tesauro. 2019. Learning to Learn without Forgetting by Maximizing Transfer and Minimizing Interference. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net. <https://openreview.net/forum?id=B1gTShAct7>
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. 2016. Prioritized Experience Replay. arXiv:1511.05952 [cs.LG] <https://arxiv.org/abs/1511.05952>
- Shagun Sodhani, Amy Zhang, and Joelle Pineau. 2021. Multi-Task Reinforcement Learning with Context-based Representations. arXiv:2102.06177 [cs.LG] <https://arxiv.org/abs/2102.06177>
- Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. 2019a. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. *CoRR* abs/1910.10897 (2019). arXiv:1910.10897 <http://arxiv.org/abs/1910.10897>
- Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. 2019b. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. In *Conference on Robot Learning (CoRL)*.

A Implementation Details

In this section, we provide more detail about our setup, specifically the tasks and agent configuration.

A.1 MetaWorld Environment Tasks

The tasks in MT10 are defined below. A visual depiction is shown in Figure 3:

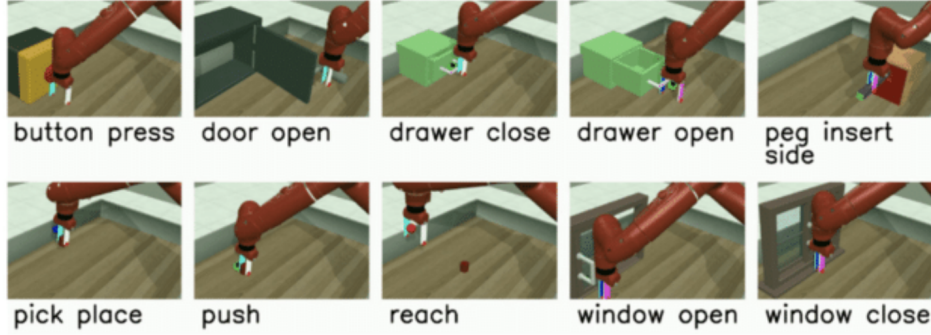


Figure 3: Visual depictions of the 10 tasks in MetaWorld’s MT10 environment.

- T0: reach-v1: Reach a goal position. Randomize the goal positions.
T1: push-v1: Push the puck to a goal. Randomize puck and goal positions.
T2: pick-place-v1: Pick and place a puck to a goal. Randomize puck and goal positions.
T3: door-open-v1: Open a door with a revolving joint. Randomize door positions.
T4: drawer-open-v1: Open a drawer. Randomize drawer positions.
T5: drawer-close-v1: Push and close a drawer. Randomize the drawer positions.
T6: button-press-topdown-v1: Press a button from the top. Randomize button positions.
T7: peg-insert-side-v1: Insert a peg sideways. Randomize peg and goal positions.
T8: window-open-v1: Push and open a window. Randomize window positions.
T9: window-close-v1: Push and close a window. Randomize window positions.

A.1.1 Training Hyperparameters

Training information:

- **Initialization Steps:** 1500
- **Total Training Steps:** 1,000,000
- **Evaluation Frequency:** Every 10,000 steps
- **Evaluation Episodes:** 10 per evaluation

Agent configuration:

- **Encoder Feature Dimension:** 50
- **Actor Network**
 - Number of Layers: 3
 - Hidden Dimension: 326
 - Optimizer: Adam
 - * Learning Rate: 0.0003
 - * Betas: (0.9, 0.9999)
- **Critic Network**
 - Number of Layers: 3
 - Hidden Dimension: 326
 - Optimizer: Adam (same as actor)
- **Task Encoder Configuration**
 - Number of Task Embeddings: 10 (used for MT10)
 - Embedding Dimension: 50
 - Hidden Dimension: 50
 - Number of Layers: 2
 - Output Dimension: 50

- **PAL (Project-and-Learn) Configuration**
 - Projection Dimension (pal_dim): 50
 - Shared Projection: true
- **GradNorm Task Weighting**
 - α : 1.0
- **Other Optimizers**
 - Alpha, Decoder, Encoder Optimizers: Adam
 - Learning Rate: 0.0003
 - Betas: (0.9, 0.9999)

B Compute Limitations

Although we were able to run dozens of experiments, we were limited by compute. Each run of MT10, took roughly 32 hours while each run of MT50 took 110 hours on a single NVIDIA RTX 3090 GPU.